

Lecture – 8

Advanced Sorting Methods –Part 2

M.G.Noel A.S Fernando (PhD)

Professor

NSBM/Dept. of Information Systems Engineering, UCSC



Quick Sort Algorithm

- Quick sort is best on the method of sorting by exchange.
- This is an improved version of bubble sort and is a lot faster, on average than the bubble sort.
- This algorithm was invented in 1960 and it is still the best sorting algorithm known for sequential computers.

Quick sort is :

- Easy to implement
- Consumes few recourses than any other sorting methods in many situations.

It works by

- partitioning the file into two sub-files
- Sorting these two sub files independently in recursive way.

Quicksort

- The quick sort algorithm also uses the divide and conquer strategy.
- The quick sort partitions the sequence by dividing it into two segments based on a selected pivot key.
- In addition, the quick sort can be implemented to work with virtual subsequences without the need for temporary storage.

Quicksort

- **Quicksort** is a Divide and Conquer algorithm.
- It picks an element as a pivot and partitions the given array around the picked pivot.
- There are many different versions of quickSort that pick pivot in different ways.
 - pick the first element as a pivot.
 - pick the last element as a pivot
 - Pick a random element as a pivot.
 - Pick median as the pivot.

Quicksort

- The quick sort is a simple recursive algorithm that can be used to sort keys stored in either an array or list.
- Given the sequence, it performs the following steps:

Quicksort

- The first key is selected as the pivot, p . The pivot value is used to partition the list into two segments or subsequences, L and U , such that L contains all keys less than the p and U contains all keys greater than or equal to p .
- The algorithm is then applied recursively to both L and U . The recursion continues until the base case is reached, which occurs when the sequence contains fewer than two keys.
- The two segments and the pivot value are merged to produce a sorted sequence.
- This is accomplished by copying the keys from segment L back into the original sequence, followed by the pivot value and then the keys from segment U .
- After this step, the pivot key will end up in its proper position within the sorted sequence.

Quicksort



Quicksort divides an array/list into two pieces. These pieces are not necessarily halves of the array. It chooses one element in the array — called the pivot (p) and rearranges the array/list elements so that

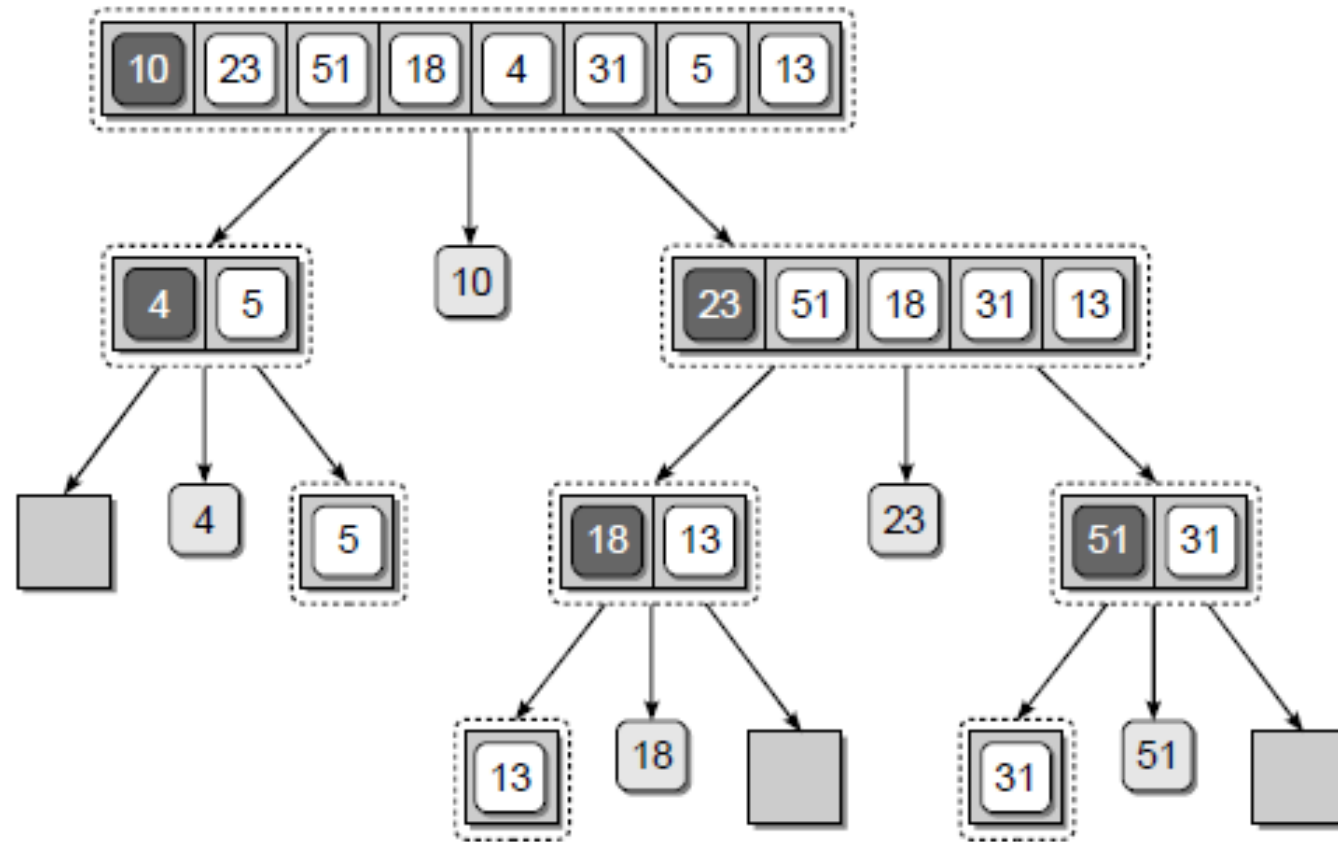
- Elements in positions before the pivot are less than the pivot
- Elements in positions after the pivot are larger than or equal to the pivot

The arrangement is called a partition of the array/list.

Partition

- Let x be an array/list and n be the number of elements in the array to be sorted.
- Choose a element a (pivot) from a specific positions within the array (e.g a can be chosen as the first element so that $a=x[0]$)
 - Suppose that the element of x are partitioned so that a is placed into position J and the following conditions are hold:
 1. Each of the elements in positions 1 through $j-1$ is less than to a
 2. each of the elements in positions $j+1$ to n is grater than of equal to a

Example



Question 2

a) Sort the following dataset using the **Quick Sort algorithm**:

25, 57, 48, 37, 12, 92, 86, 33

- **Note:** Use the **first element** as the pivot at all stages.

b) write pseudocode algorithm for the quick sort

Partition - Algorithm

- Let $a=x[lb]$ be the element whose final position is sought.
- Two references **up** and **down** are initialized to the upper and lower bounds of the sub-array respectively.
- At any point during the execution, each element in a position above **up** is greater than **a**.

Partition Algorithm

- Two references up and down are moved towards each other in the following fashion.
- Algorithm
- **Step 1:** Repeatedly increase the pointer down by one position until $x[\text{down}] > a$
- **Step 2:** Repeatedly decrease the pointer up by one position until $x[\text{up}] \leq a$
- **Step 3:** if $\text{up} > \text{down}$ then interchange **$x[\text{down}]$ with $x[\text{up}]$**
- The process is repeated until the condition in step 3 fails (ie $\text{up} \leq \text{down}$) at that point **$x[\text{up}]$** is interchanged with **$x[\text{lb}]$** (which equal to a), whose final position was sought and **j** is set to up.

Question 3

- Apply the above portion algorithm portion the following data set:
- 25 57 48 37 12 92 86 33